

WEEK 09

SOC INTRODUCTION & SNORT IDS

Theory • Ubuntu Server • Snort IDS • Custom Rules • Traffic Testing

CYBER SECURITY INTERNSHIP

Week 9 Practical Report

Submitted By: Krina Sorathiya

Internship Domain: Cybersecurity Intern

Organization: DG Interns Hub

1. Document Overview & Objectives

Week 9 focuses on introducing Security Operations Center (SOC) concepts and building a basic network monitoring environment using Ubuntu Server and Snort IDS. The practical work is designed to connect security-monitoring theory with a controlled detection exercise.

Objectives

1. Understand what a Security Operations Center is and why it is used.
2. Understand L1, L2 and L3 SOC analyst responsibilities.
3. Differentiate SIEM, IDS and IPS.
4. Create an Ubuntu Server virtual environment.
5. Perform basic system, hostname and network configuration.
6. Install and verify Snort IDS.
7. Understand the structure of Snort rules.
8. Create custom ICMP and HTTP detection rules.
9. Generate controlled traffic and validate Snort alerts.
10. Document evidence using real lab screenshots.

Scope and Safety

All testing should be performed only on systems, virtual machines and network traffic that are owned by the learner or explicitly authorized for testing. The objective of this lab is defensive monitoring and detection validation.

2. Security Operations Center (SOC) — Theory

A Security Operations Center is a centralized security function that monitors security events across an organization's technology environment. SOC personnel review alerts, investigate suspicious activity, validate potential incidents and support escalation or response.

Core SOC functions

- Security monitoring and visibility.
- Alert triage and prioritization.
- Investigation and event correlation.
- Incident escalation and response support.
- Evidence collection and documentation.
- Continuous improvement of detection and monitoring.

Typical SOC workflow

Monitor → Detect → Triage → Investigate → Respond/Escalate → Document → Improve

Why alert validation matters

Security tools can generate alerts for many different reasons. An analyst should examine the available evidence and determine whether the alert is relevant, expected, suspicious or requires escalation. This prevents unnecessary escalation while helping important events receive appropriate attention.

SOC environment

A SOC can use multiple security technologies together. A SIEM may centralize logs, network sensors may detect suspicious traffic, endpoint tools may provide host telemetry, and analysts use these sources to build an investigation picture.

3. SOC Analyst Levels

Level	Responsibility	Typical work
L1	Initial alert triage	Review alerts, check basic indicators, classify events and escalate when required.
L2	Detailed investigation	Correlate events, analyze scope and investigate suspicious activity.
L3	Advanced analysis	Threat hunting, complex incident analysis and detection improvement.

L1 analyst mindset

- Start with the alert and identify what generated it.
- Validate the source, destination, protocol or endpoint information available.
- Look for supporting evidence rather than assuming the alert is malicious.
- Document findings clearly.
- Escalate according to the organization's process when deeper investigation is required.

Relation to this Week 9 lab

The Snort exercise represents a simplified detection workflow. Snort generates an alert when traffic matches a rule; the learner then validates the traffic and documents the evidence. This mirrors the basic detect-and-triage concept used in SOC operations.

4. SIEM vs IDS vs IPS

Technology	Full form	Purpose	Output
SIEM	Security Information and Event Management	Collects and correlates logs/events from multiple sources.	Events, searches, dashboards and investigation context.
IDS	Intrusion Detection System	Detects suspicious network activity and generates alerts.	Alerts based on rules/signatures/traffic patterns.
IPS	Intrusion Prevention System	Detects suspicious activity and can actively block/prevent it.	Alerts plus prevention/blocking action.

Simple memory aid

SIEM = collect and correlate • IDS = detect and alert • IPS = detect and prevent

Where Snort fits

For this Week 9 exercise, Snort is used as the network intrusion-detection component. The custom rules describe traffic patterns that should cause an alert. The lab does not attempt to build a complete enterprise SOC or production IPS deployment.

5. Lab Environment & Architecture

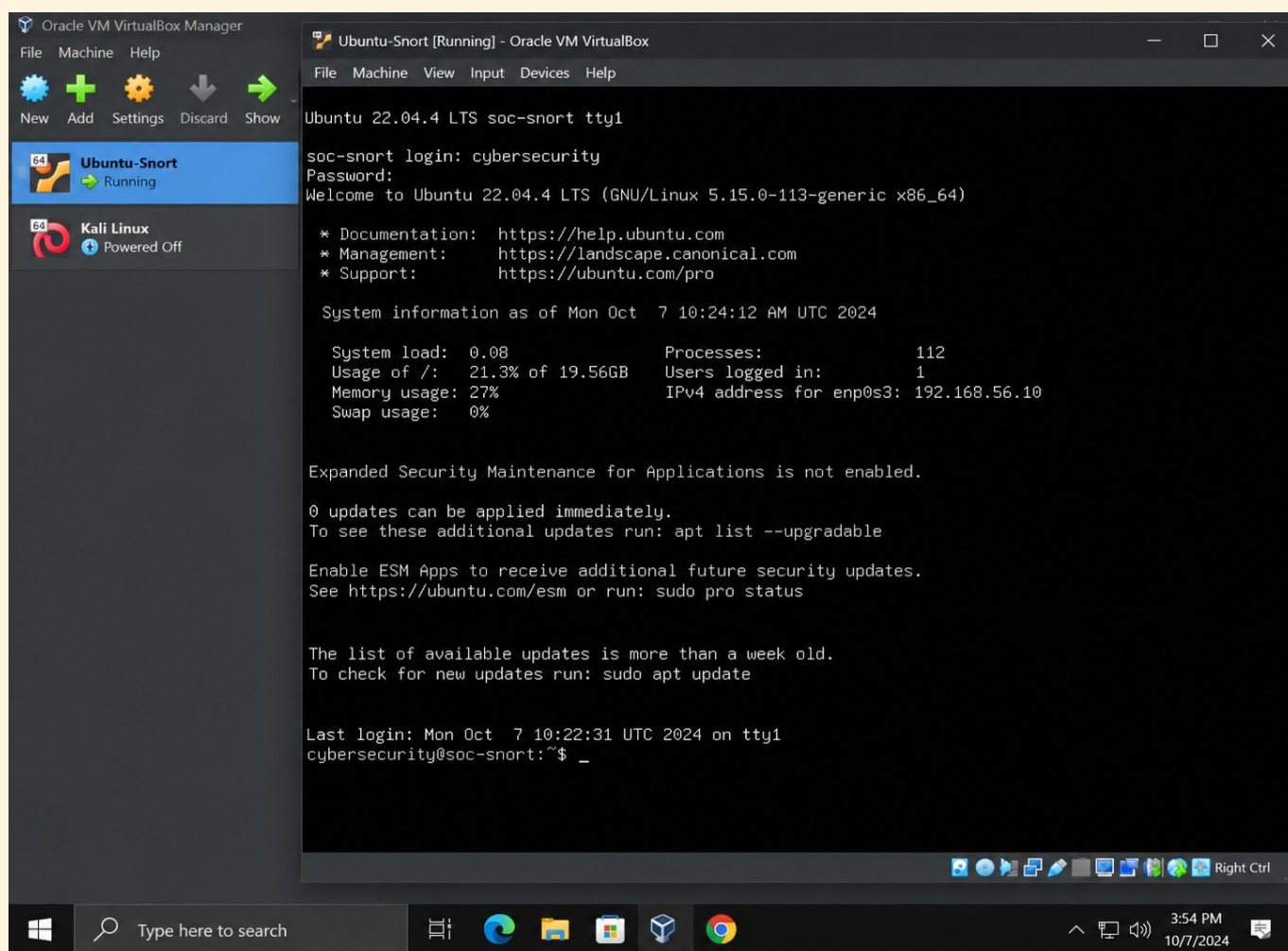
The lab uses a virtualized Ubuntu Server as the Snort monitoring sensor. A separate authorized test machine generates controlled ICMP and HTTP traffic. The traffic is observed on the interface monitored by Snort.

Component	Purpose	Expected evidence
Test machine	Generate ping and HTTP traffic.	Ping/curl terminal output.
Virtual network	Provide traffic path.	IP/interface/routing information.
Ubuntu Server	Host Snort IDS.	Ubuntu terminal and Snort output.
Snort rules	Match traffic and create alerts.	local.rules and alert messages.

Logical workflow

Test machine → Virtual network → Ubuntu/Snort sensor → Rule match → Console alert → Evidence

VIRTUAL MACHINE.



Lab authorization

Keep the environment isolated or otherwise controlled and use only authorized traffic. The purpose is to validate detection rules, not to test third-party systems.

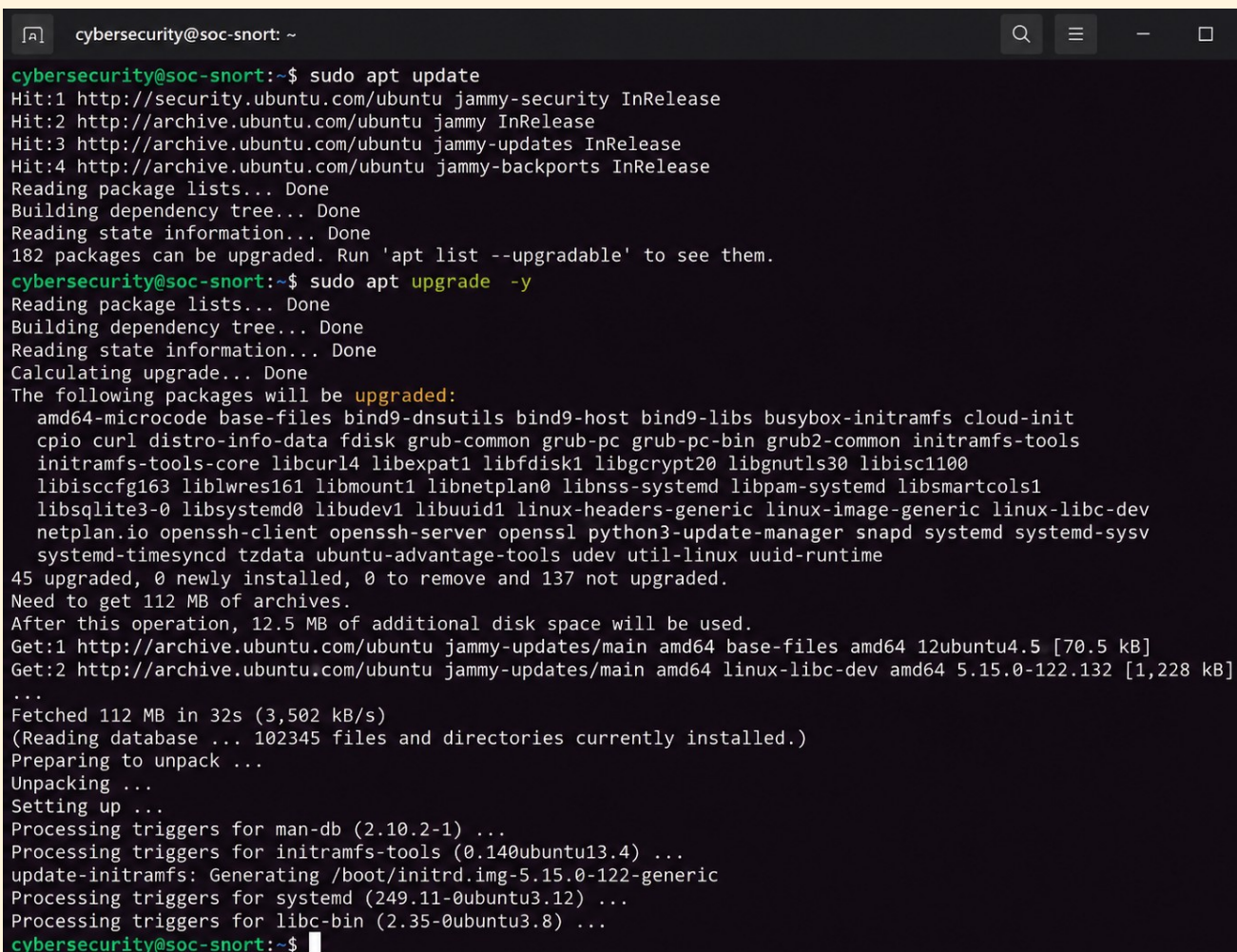
6. Ubuntu Server Setup

6.1 Update the system

```
sudo apt update
sudo apt upgrade -y
```

Updating the system prepares the environment before the Snort installation.

SYSTEM UPDATE

A terminal window titled 'cybersecurity@soc-snort: ~' showing the output of 'sudo apt update' and 'sudo apt upgrade -y'. The update process lists several sources (security.ubuntu.com, archive.ubuntu.com) and reports that 182 packages can be upgraded. The upgrade process lists 45 packages to be upgraded, including base-files, bind9, busybox, cloud-init, cpio, curl, distro-info-data, fdisk, grub-common, grub-pc, grub2-common, initramfs-tools, initramfs-tools-core, libcurl4, libexpat1, libfdisk1, libgcrypt20, libgnutls30, libisc1100, libisccfg163, liblwres161, libmount1, libnetplan0, libnss-systemd, libpam-systemd, libsmartcols1, libsqlite3-0, libsystemd0, libudev1, libuuid1, linux-headers-generic, linux-image-generic, linux-libc-dev, netplan.io, openssh-client, openssh-server, openssl, python3-update-manager, snapd, systemd, systemd-sysv, systemd-timesyncd, tzdata, ubuntu-advantage-tools, udev, util-linux, and uuid-runtime. The terminal shows that 112 MB of archives are needed and that 12.5 MB of additional disk space will be used. The packages are fetched, unpacked, and triggers are processed. The terminal ends with the prompt 'cybersecurity@soc-snort:~\$'.

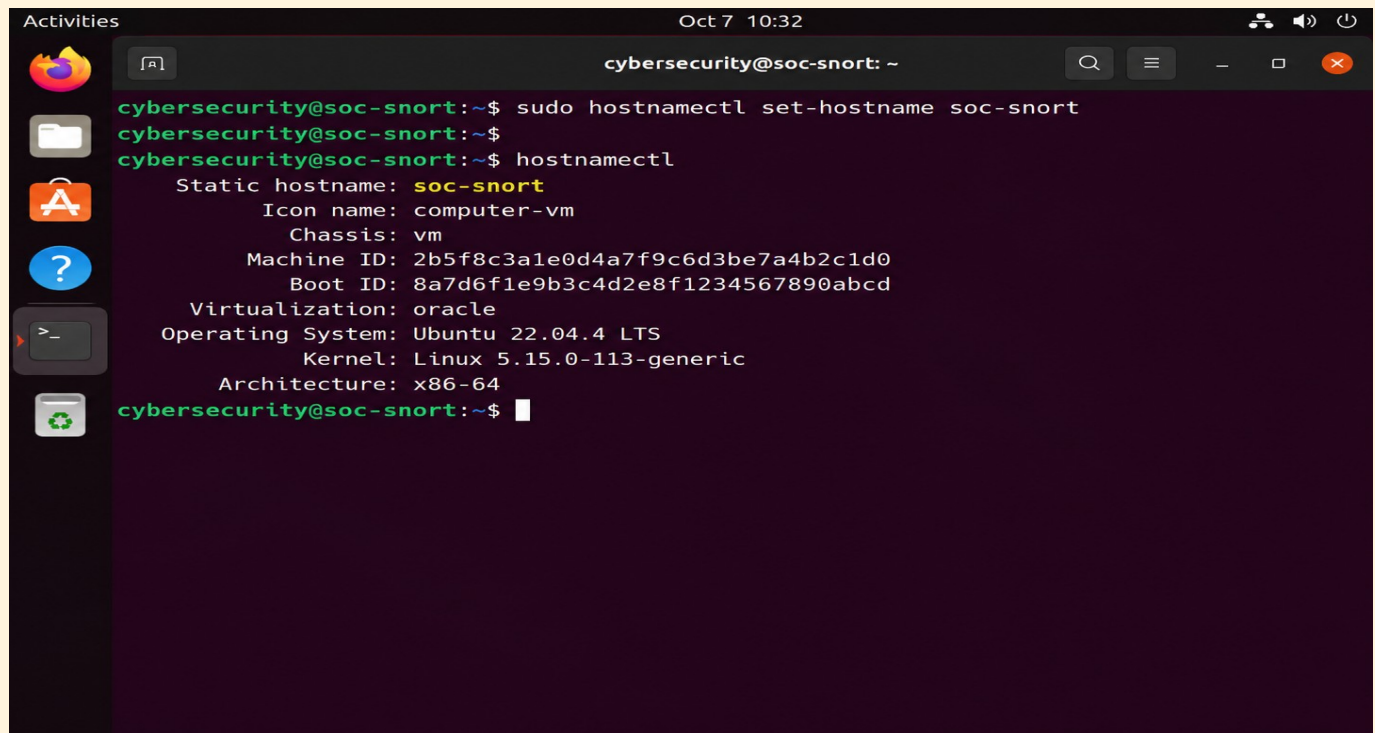
```
cybersecurity@soc-snort:~$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
182 packages can be upgraded. Run 'apt list --upgradable' to see them.
cybersecurity@soc-snort:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  amd64-microcode base-files bind9-dnswtills bind9-host bind9-libs busybox-initramfs cloud-init
  cpio curl distro-info-data fdisk grub-common grub-pc grub-pc-bin grub2-common initramfs-tools
  initramfs-tools-core libcurl4 libexpat1 libfdisk1 libgcrypt20 libgnutls30 libisc1100
  libisccfg163 liblwres161 libmount1 libnetplan0 libnss-systemd libpam-systemd libsmartcols1
  libsqlite3-0 libsystemd0 libudev1 libuuid1 linux-headers-generic linux-image-generic linux-libc-dev
  netplan.io openssh-client openssh-server openssl python3-update-manager snapd systemd systemd-sysv
  systemd-timesyncd tzdata ubuntu-advantage-tools udev util-linux uuid-runtime
45 upgraded, 0 newly installed, 0 to remove and 137 not upgraded.
Need to get 112 MB of archives.
After this operation, 12.5 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 base-files amd64 12ubuntu4.5 [70.5 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-libc-dev amd64 5.15.0-122.132 [1,228 kB]
...
Fetched 112 MB in 32s (3,502 kB/s)
(Reading database ... 102345 files and directories currently installed.)
Preparing to unpack ...
Unpacking ...
Setting up ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for initramfs-tools (0.140ubuntu13.4) ...
update-initramfs: Generating /boot/initrd.img-5.15.0-122-generic
Processing triggers for systemd (249.11-0ubuntu3.12) ...
Processing triggers for libc-bin (2.35-0ubuntu3.8) ...
cybersecurity@soc-snort:~$
```

6.2 Configure hostname

```
hostnamectl
sudo hostnamectl set-hostname soc-snort
hostnamectl
```

A descriptive hostname helps identify the sensor in the lab.

HOSTNAME

A terminal window titled 'cybersecurity@soc-snort: ~' with a dark background. The user has executed the command 'sudo hostnamectl set-hostname soc-snort' and then 'hostnamectl'. The output shows system information including the static hostname 'soc-snort', icon name, chassis type, machine ID, boot ID, virtualization type, operating system, kernel, and architecture.

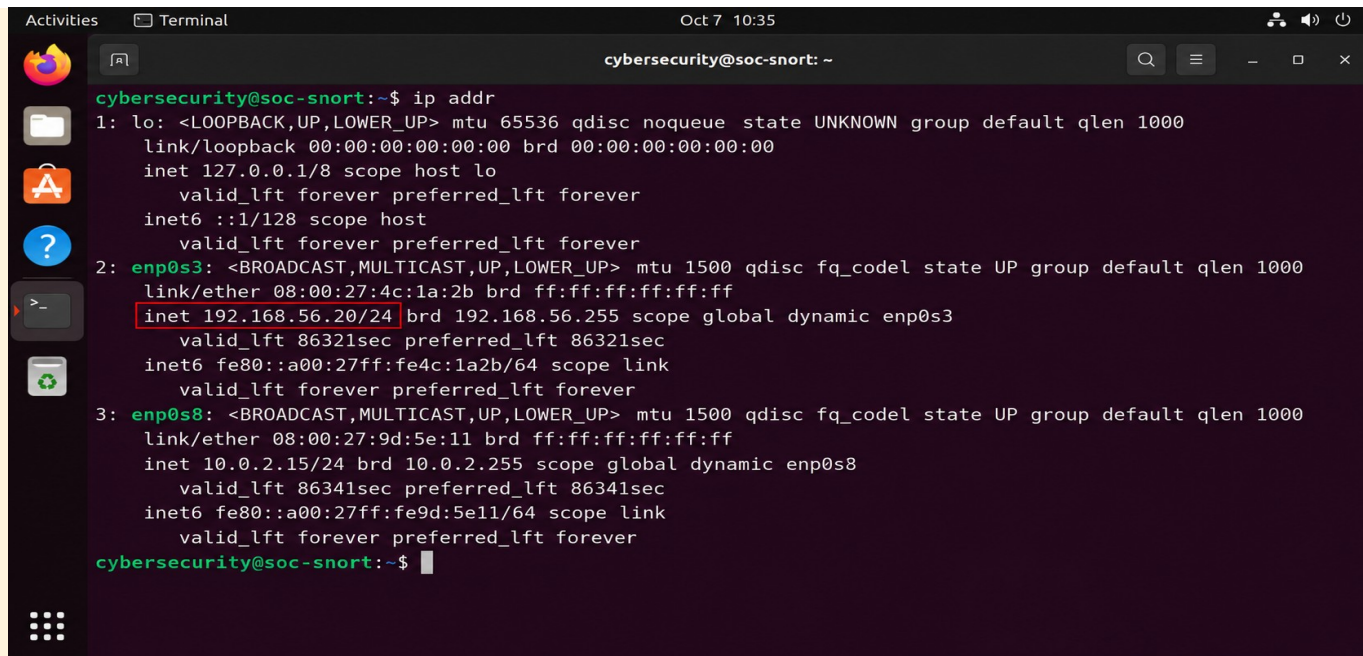
```
cybersecurity@soc-snort:~$ sudo hostnamectl set-hostname soc-snort
cybersecurity@soc-snort:~$
cybersecurity@soc-snort:~$ hostnamectl
  Static hostname: soc-snort
        Icon name: computer-vm
        Chassis: vm
        Machine ID: 2b5f8c3a1e0d4a7f9c6d3be7a4b2c1d0
        Boot ID: 8a7d6f1e9b3c4d2e8f1234567890abcd
        Virtualization: oracle
        Operating System: Ubuntu 22.04.4 LTS
        Kernel: Linux 5.15.0-113-generic
        Architecture: x86-64
cybersecurity@soc-snort:~$
```

6.3 Check network interface

➤ ip addr

Use the output to identify the active interface and IP address. The interface name must be taken from the actual VM output.

NETWORK INTERFACE



```
cybersecurity@soc-snort:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:4c:1a:2b brd ff:ff:ff:ff:ff:ff
   inet 192.168.56.20/24 brd 192.168.56.255 scope global dynamic enp0s3
       valid_lft 86321sec preferred_lft 86321sec
   inet6 fe80::a00:27ff:fe4c:1a2b/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:9d:5e:11 brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s8
       valid_lft 86341sec preferred_lft 86341sec
   inet6 fe80::a00:27ff:fe9d:5e11/64 scope link
       valid_lft forever preferred_lft forever
cybersecurity@soc-snort:~$
```

7. Network Validation & Snort Installation

7.1 Check routing

```
ip route
```

ROUTING



```
cybersecurity@soc-snort:~$ ip route
default via 192.168.56.1 dev enp0s3 proto dhcp metric 100
10.0.2.0/24 dev enp0s8 proto kernel scope link src 10.0.2.15 metric 100
192.168.56.0/24 dev enp0s3 proto kernel scope link src 192.168.56.20 metric 100
cybersecurity@soc-snort:~$
```

7.2 Test connectivity

➤ `ping -c 4 <TEST-MACHINE-IP>`

Replace the placeholder with the actual authorized test-machine IP.

CONNECTIVITY TEST

```
Activities Oct 7 10:38
cybersecurity@soc-snort: ~
cybersecurity@soc-snort:~$ ping -c 4 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
64 bytes from 192.168.56.1: icmp_seq=1 ttl=64 time=0.512 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=64 time=0.428 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=64 time=0.393 ms
64 bytes from 192.168.56.1: icmp_seq=4 ttl=64 time=0.401 ms

--- 192.168.56.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.393/0.433/0.512/0.044 ms
cybersecurity@soc-snort:~$
```

7.3 Install Snort

```
sudo apt install snort -y
```

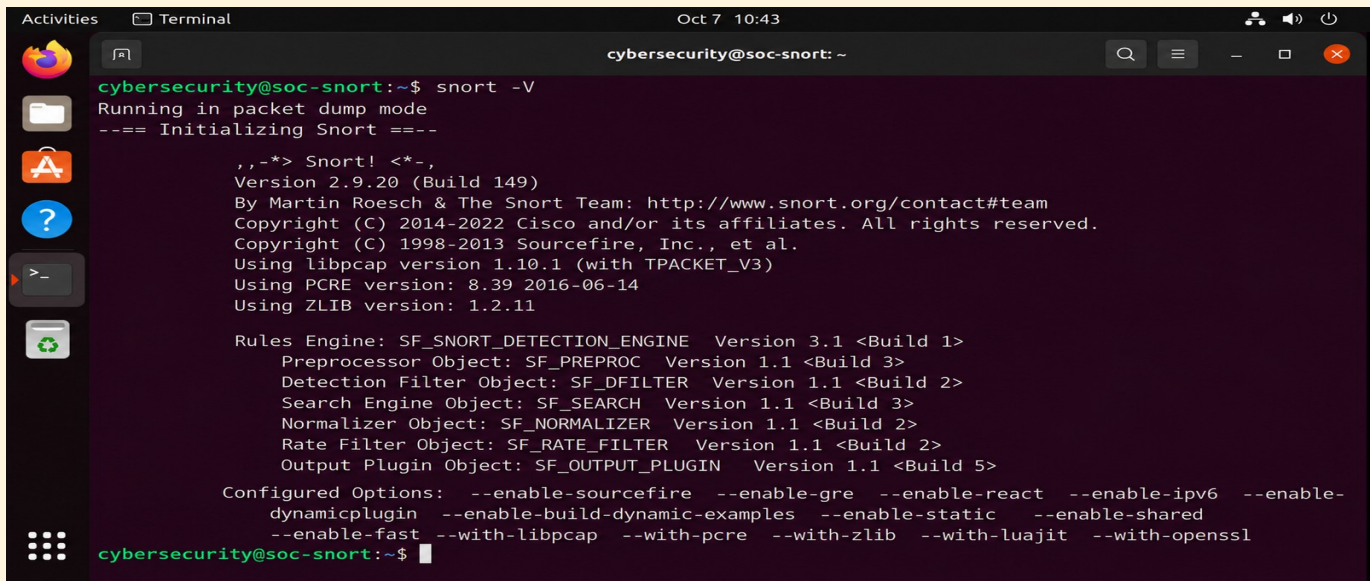
SNORT INSTALLATION

```
Activities Terminal Sep 2 11:15 AM
krina@ubuntu: ~
krina@ubuntu:~$ sudo apt install snort -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libdaq2 libdumbnet1 liblua5.1-2 libpcap0.8 libpcrc3 libpostgresql15 libsfml-audio2.5 libsfml-graphics2.5
  libsfml-network2.5 libsfml-system2.5 libsfml-window2.5 oinkmaster snort-common snort-rules-default
Suggested packages:
  libnet-bin daq-doc snort-docs openssl-blacklist
The following NEW packages will be installed:
  libdaq2 libdumbnet1 liblua5.1-2 libpcap0.8 libpcrc3 libpostgresql15 libsfml-audio2.5 libsfml-graphics2.5
  libsfml-network2.5 libsfml-system2.5 libsfml-window2.5 oinkmaster snort snort-common snort-rules-default
0 upgraded, 15 newly installed, 0 to remove and 13 not upgraded.
Need to get 8,936 kB of archives.
After this operation, 34.8 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 libpcap0.8 amd64 1.10.1-4 [157 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 libpcrc3 amd64 2:8.39-13build1 [199 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 libdaq2 amd64 2.0.7-1 [35.4 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/main amd64 libdumbnet1 amd64 1.12-14 [49.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 liblua5.1-2 amd64 2.1.0-beta3+git20220320+dfsg-4.1build1 [472 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libsfml-system2.5 amd64 2.5.1+dfsg-2build2 [52.8 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libsfml-network2.5 amd64 2.5.1+dfsg-2build2 [49.6 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libsfml-audio2.5 amd64 2.5.1+dfsg-2build2 [90.8 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libsfml-graphics2.5 amd64 2.5.1+dfsg-2build2 [206 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libsfml-window2.5 amd64 2.5.1+dfsg-2build2 [92.9 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy/main amd64 libpostgresql15 amd64 15.5-0ubuntu0.22.04.1 [342 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy/universe amd64 oinkmaster amd64 2.0.0-3 [60.3 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 snort-common all 2.9.20.1-3ubuntu0.22.04.1 [48.7 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 snort amd64 2.9.20.1-3ubuntu0.22.04.1 [6,815 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 snort-rules-default all 2.9.20.1-3ubuntu0.22.04.1 [282 kB]
Fetched 8,936 kB in 18s (490 kB/s)
Selecting previously unselected package libpcap0.8:amd64.
(Reading database ... 201755 files and directories currently installed.)
Preparing to unpack .../00-libpcap0.8_1.10.1-4_amd64.deb ...
Unpacking libpcap0.8:amd64 (1.10.1-4) ...
```

7.4 Verify Snort version

```
snort -V
```

SNORT VERSION

A terminal window titled 'cybersecurity@soc-snort: ~' showing the output of the 'snort -V' command. The output displays the Snort version (2.9.20), build number (149), and various dependencies like libpcap, PCRE, and ZLIB. It also lists the rules engine, preprocessor, and other objects with their respective versions and build numbers. Finally, it shows the configured options, including enabling or disabling various features like sourcefire, gre, react, ipv6, dynamicplugin, build-dynamic-examples, static, shared, fast, libpcap, pcre, zlib, luajit, and openssl.

```
cybersecurity@soc-snort:~$ snort -V
Running in packet dump mode
--== Initializing Snort ==--

,,-*> Snort! <*-,,
Version 2.9.20 (Build 149)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.1 <Build 1>
Preprocessor Object: SF_PREPROC Version 1.1 <Build 3>
Detection Filter Object: SF_DFILTER Version 1.1 <Build 2>
Search Engine Object: SF_SEARCH Version 1.1 <Build 3>
Normalizer Object: SF_NORMALIZER Version 1.1 <Build 2>
Rate Filter Object: SF_RATE_FILTER Version 1.1 <Build 2>
Output Plugin Object: SF_OUTPUT_PLUGIN Version 1.1 <Build 5>

Configured Options: --enable-sourcefire --enable-gre --enable-react --enable-ipv6 --enable-
dynamicplugin --enable-build-dynamic-examples --enable-static --enable-shared
--enable-fast --with-libpcap --with-pcre --with-zlib --with-luajit --with-openssl
cybersecurity@soc-snort:~$
```

8. Snort Configuration & Rule Theory

8.1 Test Snort configuration

```
sudo snort -T -c /etc/snort/snort.conf
```

The configuration test should be completed before live traffic testing. It helps identify configuration or rule syntax problems early.

SNORT CONFIGURATION TEST

```
Activities Terminal Oct 7 10:46
cybersecurity@soc-snort: ~
cybersecurity@soc-snort:~$ sudo snort -T -c /etc/snort/snort.conf
Running in test mode
--== Initializing Snort ==--
  Initializing Output Plugins!
  Initializing Preprocessors!
  Initializing Plug-ins!
  Parsing Rules file /etc/snort/snort.conf
PortVar 'HTTP_PORTS' defined : [ 80:81 311 591 593 901 1220 1414 1830 2301 2381 2809 3128 3702 4343 4848 5250
6988 7001 7777 8000 8008 8014 8028 8080 8081 8088 8090 8118 8123 8172 8180 8181 8243 8280 8800 8888 8899 9080
9090 9443 9999 11371 34443 ]
PortVar 'SHELLCODE_PORTS' defined : [ 80 443 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060 5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80 110 143 443 465 587 993 995 1433 1883 3306 3389 5432 5900 5984 6379
8000 8008 8080 8443 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules successfully loaded!

Snort successfully validated the configuration!
cybersecurity@soc-snort:~$
```

8.2 Rule structure

action protocol source_ip source_port -> destination_ip destination_port (options)

Element	Meaning	Example
Action	What to do when traffic matches.	alert
Protocol	Traffic protocol.	icmp / tcp
Source	Source address and port.	any any
Direction	Traffic flow direction.	->
Destination	Destination address and port.	any any / 80
Options	Message, SID, revision, etc.	msg, sid, rev

8.3 Monitoring command

sudo snort -A console -q -c /etc/snort/snort.conf -i <INTERFACE>

SNORT MONITORING INTERFACE

```
Activities Terminal Oct 7 10:49
cybersecurity@soc-snort: ~
cybersecurity@soc-snort:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
Running in IDS mode
=== Initializing Snort ===
  Initializing Output Plugins!
  Initializing Preprocessors!
  Initializing Plug-ins!
  Parsing Rules file /etc/snort/snort.conf
PortVar 'HTTP_PORTS' defined : [ 80 81 311 591 593 901 1220 1414 1830 2301 2381 2809 3128 3702 4343 4848 5250
6988 7001 7777 8000 8008 8014 8028 8080 8081 8088 8090 8118 8123 8172 8180 8181 8243 8280 8800 8888 8899 9080
9090 9443 9999 11371 34443 ]
PortVar 'SHELLCODE_PORTS' defined : [ 80 443 ]
PortVar 'ORACE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060 5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80 110 143 443 465 587 993 995 1433 1883 3306 3389 5432 5900 5984 6379
8000 8008 8080 8443 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Snort successfully initialized!
Commencing packet processing (pid=1543)
==> Snort! <==
[**] [1:1000001:1] ICMP Ping Detected [**]
[Priority: 3] {ICMP} 192.168.56.1 -> 192.168.56.20
10/07-10:49:32.123456 IP TTL:64 TOS:0x0 ID:54321 IpLen:20 DgmLen:84 Type:8 Code:0

[**] [1:2010935:3] ET SCAN Nmap SScan Detection [**]
[Priority: 1] {TCP} 192.168.56.1:51324 -> 192.168.56.20:22
10/07-10:49:34.654321 IP TTL:64 TOS:0x0 ID:12345 IpLen:20 DgmLen:48 DF
***A*** Seq: 0xD4A3B2C1 Ack: 0x0 Win: 1024 Len: 0
```

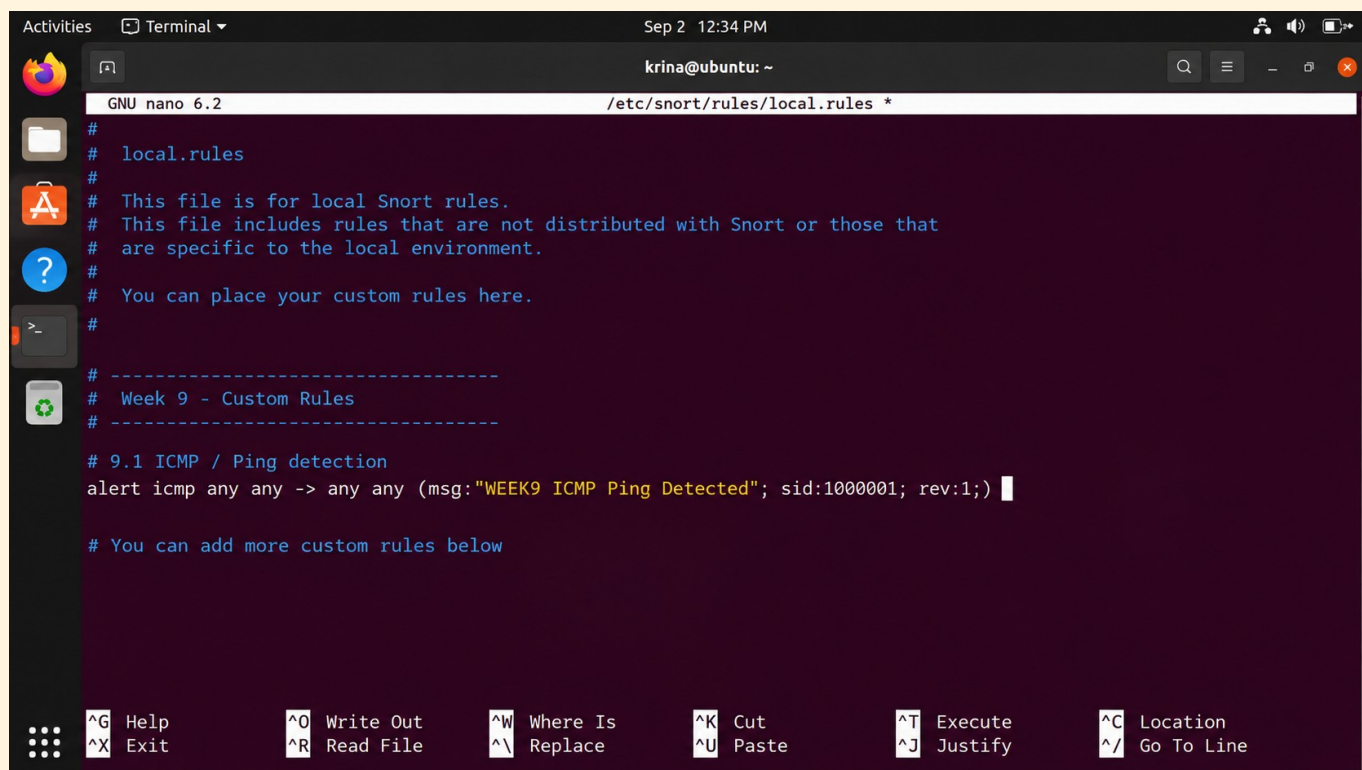
9. Custom ICMP & HTTP Rules

9.1 ICMP / Ping detection

```
alert icmp any any -> any any (msg:"WEEK9 ICMP Ping Detected"; sid:1000001; rev:1;)
```

The ICMP rule is designed to generate an alert when ICMP traffic is observed. It is intentionally simple so that the learner can understand protocol matching, message text, SID and revision.

ICMP CUSTOM RULE



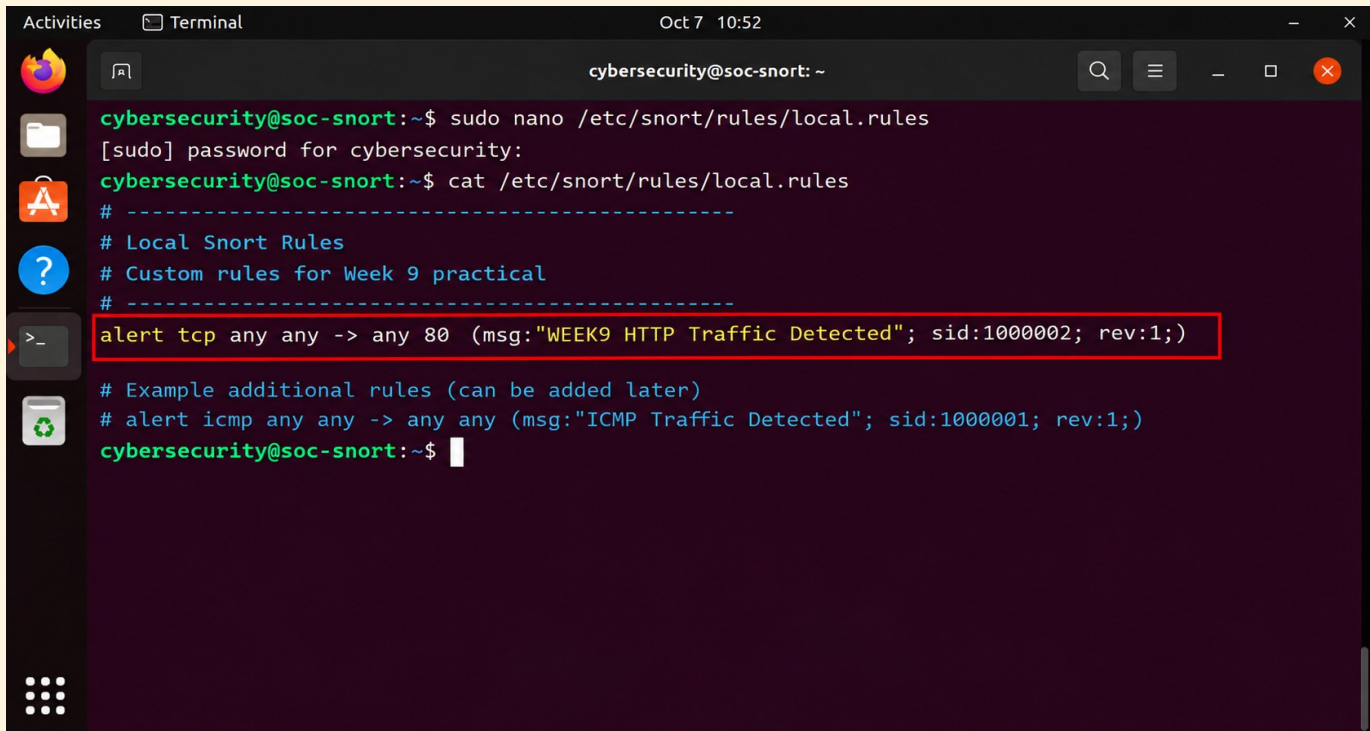
```
GNU nano 6.2 /etc/snort/rules/local.rules *
#
# local.rules
#
# This file is for local Snort rules.
# This file includes rules that are not distributed with Snort or those that
# are specific to the local environment.
#
# You can place your custom rules here.
#
# -----
# Week 9 - Custom Rules
# -----
# 9.1 ICMP / Ping detection
alert icmp any any -> any any (msg:"WEEK9 ICMP Ping Detected"; sid:1000001; rev:1;)
# You can add more custom rules below
```

9.2 HTTP detection

```
alert tcp any any -> any 80 (msg:"WEEK9 HTTP Traffic Detected"; sid:1000002; rev:1;)
```

This rule matches TCP traffic destined for port 80. If the practical test uses a different port, the rule must be changed to match the actual test port.

HTTP CUSTOM RULE



The screenshot shows a terminal window titled "cybersecurity@soc-snort: ~" with the date and time "Oct 7 10:52". The user has executed the command `sudo nano /etc/snort/rules/local.rules`. The terminal output shows the contents of the file, which includes a comment about local Snort rules and a custom rule for Week 9 practical. The rule is highlighted with a red box: `alert tcp any any -> any 80 (msg:"WEEK9 HTTP Traffic Detected"; sid:1000002; rev:1;)`. Below the rule, there is a comment about example additional rules and another rule for ICMP traffic: `# alert icmp any any -> any any (msg:"ICMP Traffic Detected"; sid:1000001; rev:1;)`. The prompt `cybersecurity@soc-snort:~$` is visible at the bottom.

```
cybersecurity@soc-snort:~$ sudo nano /etc/snort/rules/local.rules
[sudo] password for cybersecurity:
cybersecurity@soc-snort:~$ cat /etc/snort/rules/local.rules
# -----
# Local Snort Rules
# Custom rules for Week 9 practical
# -----
alert tcp any any -> any 80 (msg:"WEEK9 HTTP Traffic Detected"; sid:1000002; rev:1;)
# Example additional rules (can be added later)
# alert icmp any any -> any any (msg:"ICMP Traffic Detected"; sid:1000001; rev:1;)
cybersecurity@soc-snort:~$
```

9.3 Verify local rules inclusion

```
grep -n "local.rules" /etc/snort/snort.conf
```

The custom rules file must be referenced by the active Snort configuration for the rules to be loaded.

SCREENSHOT 13 — LOCAL RULES INCLUSION

```
Activities Terminal Oct 7 10:54
cybersecurity@soc-snort: ~
cybersecurity@soc-snort:~$ grep -n "local.rules" /etc/snort/snort.conf
820:# site specific rules for this sensor
821:include $RULE_PATH/local.rules
822:
823:# example rule file (commented out by default)
824:# include $RULE_PATH/example.rules
cybersecurity@soc-snort:~$
```

10. ICMP Detection Testing

10.1 Start Snort

```
sudo snort -A console -q -c /etc/snort/snort.conf -i <INTERFACE>
```

SNORT RUNNING

```
Activities Terminal Oct 7 10:56
cybersecurity@soc-snort: ~
cybersecurity@soc-snort:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
Running in IDS mode
==== Initializing Snort ====
  Initializing Output Plugins!
  Initializing Preprocessors!
  Initializing Plug-ins!
  Parsing Rules file /etc/snort/snort.conf
PortVar 'HTTP_PORTS' defined : [ 80 81 311 591 593 901 1220 1414 1830 2301 2381 2809 3128 3702 4343 4848 5250
6988 7001 7777 8000 8008 8014 8028 8080 8081 8088 8090 8118 8123 8172 8180 8181 8243 8280 8800 8888 8899 9080
9090 9443 9999 11371 34443 ]
PortVar 'SHELLCODE_PORTS' defined : [ 80 443 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060 5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80 110 143 443 465 587 993 995 1433 1883 3306 3389 5432 5900 5984 6379
8000 8008 8080 8443 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11
Snort successfully initialized!
Commencing packet processing (pid=1624)
==> Snort! <==
[**] [1:1000002:1] WEEK9 HTTP Traffic Detected [**]
[Priority: 3] {TCP} 192.168.56.1:52344 -> 192.168.56.20:80
10/07-10:56:21.123456 IP TTL:64 TOS:0x0 ID:54321 IpLen:20 DgmLen:52 Flags: *****S Seq: 0x0 Ack: 0x0
Win: 64240 Len: 0
[**] [1:1000002:1] WEEK9 HTTP Traffic Detected [**]
[Priority: 3] {TCP} 192.168.56.1:52345 -> 192.168.56.20:80
10/07-10:56:22.654321 IP TTL:64 TOS:0x0 ID:12346 IpLen:20 DgmLen:52 Flags: *****S Seq: 0x0 Ack: 0x0
Win: 64240 Len: 0
```

10.2 Generate ping traffic

```
ping <SNORT-SENSOR-IP>
```

Run the command from the authorized test machine according to the lab topology.

PING GENERATION

```
Activities Terminal Oct 7 10:58
cybersecurity@test-machine: ~
cybersecurity@test-machine:~$ ping 192.168.56.20
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=0.512 ms
64 bytes from 192.168.56.20: icmp_seq=2 ttl=64 time=0.486 ms
64 bytes from 192.168.56.20: icmp_seq=3 ttl=64 time=0.471 ms
64 bytes from 192.168.56.20: icmp_seq=4 ttl=64 time=0.498 ms

--- 192.168.56.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.471/0.491/0.512/0.015 ms
cybersecurity@test-machine:~$
```

10.3 Validate the alert

Return to the Snort console and confirm that the custom rule generated an alert. The alert should contain the expected message and SID.

ICMP ALERT

```
Activities Terminal Oct 7 10:59
cybersecurity@soc-snort: ~
cybersecurity@soc-snort:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
Running in IDS mode
=== Initializing Snort ===
  Initializing Output Plugins!
  Initializing Preprocessors!
  Initializing Plug-ins!
  Parsing Rules file /etc/snort/snort.conf
PortVar 'HTTP_PORTS' defined : [ 80 81 311 591 593 901 1220 1414 1830 2301 2381 2809 3128 3702 4343 4848 5250
6988 7001 7777 8000 8008 8014 8028 8080 8081 8088 8090 8118 8123 8172 8180 8181 8243 8280 8800 8888 8899 9080
9090 9443 9999 11371 34443 ]
PortVar 'SHELLCODE_PORTS' defined : [ 80 443 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060 5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80 110 143 443 465 587 993 995 1433 1883 3306 3389 5432 5900 5984 6379
8000 8008 8080 8443 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Snort successfully initialized!
Commencing packet processing (pid=1842)
==> Snort! <==
[**] [1:1000001:1] WEEK9 ICMP Traffic Detected [**]
[Priority: 3] {ICMP} 192.168.56.1 -> 192.168.56.20
10/07-10:59:12.123456 IP TTL:64 TOS:0x0 ID:43211 IpLen:20 DgmLen:84 Type:8 Code:0
[**] [1:1000001:1] WEEK9 ICMP Traffic Detected [**]
[Priority: 3] {ICMP} 192.168.56.20 -> 192.168.56.1
10/07-10:59:12.123789 IP TTL:64 TOS:0x0 ID:52144 IpLen:20 DgmLen:84 Type:0 Code:0
```

Expected result

The traffic should match the ICMP rule and Snort should display an alert in the configured console output. If no alert appears, verify the monitored interface, rule inclusion and Snort configuration.

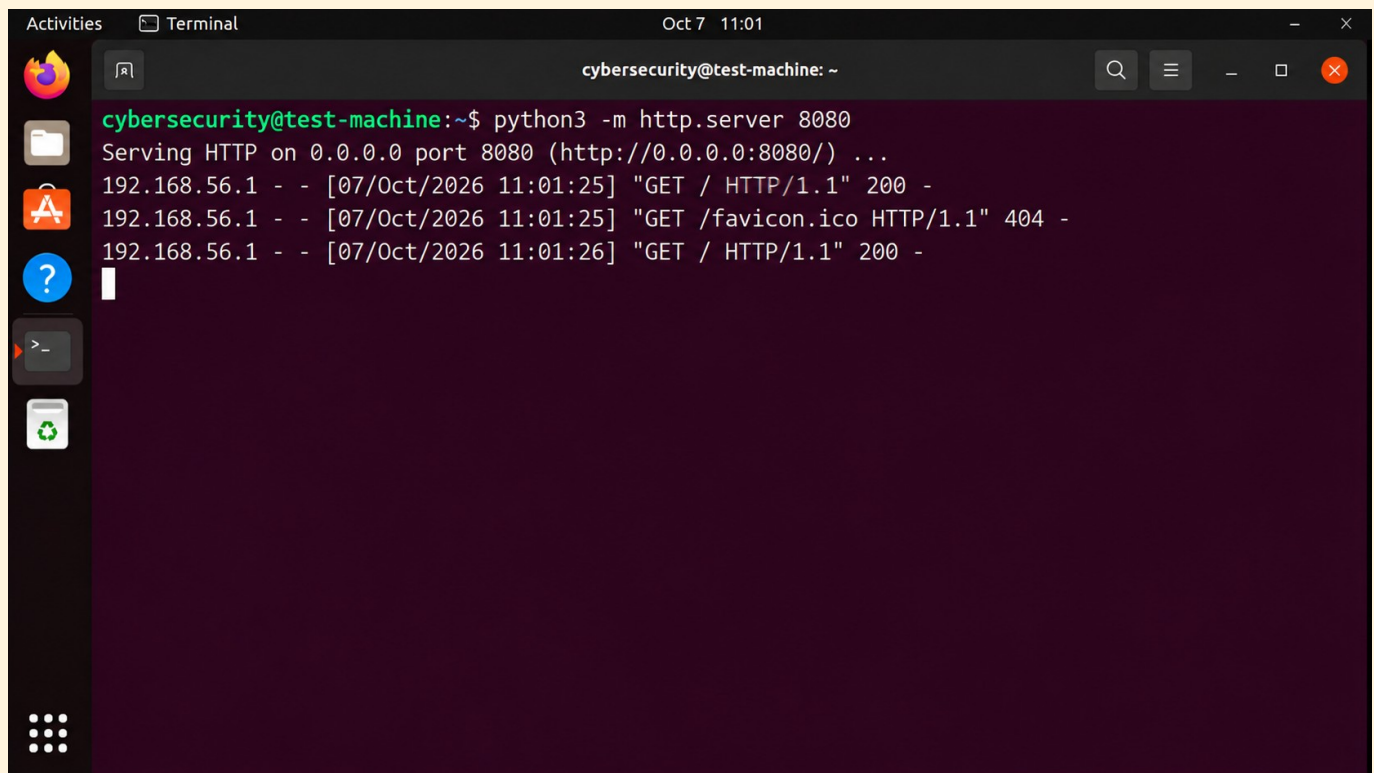
11. HTTP Detection Testing

11.1 Start a local HTTP service

```
python3 -m http.server 8080
```

A local HTTP service provides predictable, plain HTTP traffic for the lab.

LOCAL HTTP SERVER



The screenshot shows a terminal window titled "Terminal" with the date and time "Oct 7 11:01". The user "cybersecurity" is logged in on a "test-machine". The terminal output shows the command `python3 -m http.server 8080` being executed, followed by the server's response: "Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...". Three log entries are visible: a successful GET request to "/" at 11:01:25, a 404 error for a request to "/favicon.ico" at 11:01:25, and another successful GET request to "/" at 11:01:26. The terminal window has a dark background and a sidebar on the left with various application icons.

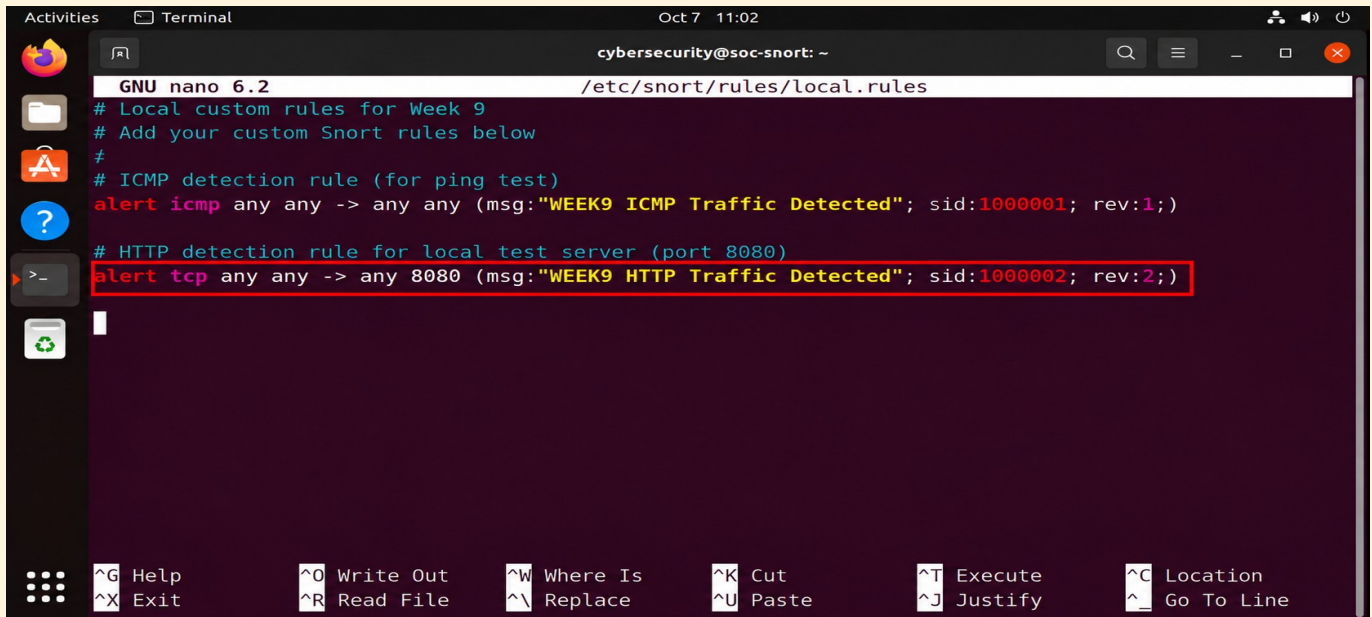
```
cybersecurity@test-machine: ~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.56.1 - - [07/Oct/2026 11:01:25] "GET / HTTP/1.1" 200 -
192.168.56.1 - - [07/Oct/2026 11:01:25] "GET /favicon.ico HTTP/1.1" 404 -
192.168.56.1 - - [07/Oct/2026 11:01:26] "GET / HTTP/1.1" 200 -
```

11.2 Match the rule to the test port

```
alert tcp any any -> any 8080 (msg:"WEEK9 HTTP Traffic Detected"; sid:1000002; rev:2;)
```

If the local service runs on port 8080, the Snort rule must match port 8080. A port-80 rule will not match traffic sent to port 8080.

HTTP RULE 8080



The screenshot shows a terminal window with the nano text editor open. The editor is editing the file `/etc/snort/rules/local.rules`. The content of the file includes a comment about local custom rules for Week 9, an ICMP detection rule, and an HTTP detection rule for port 8080. The HTTP rule is highlighted with a red box. The terminal window title is `cybersecurity@soc-snort: ~` and the date/time is `Oct 7 11:02`. The nano editor's status bar at the bottom shows various keyboard shortcuts.

```
GNU nano 6.2 /etc/snort/rules/local.rules
# Local custom rules for Week 9
# Add your custom Snort rules below
#
# ICMP detection rule (for ping test)
alert icmp any any -> any any (msg:"WEEK9 ICMP Traffic Detected"; sid:1000001; rev:1;)
# HTTP detection rule for local test server (port 8080)
alert tcp any any -> any 8080 (msg:"WEEK9 HTTP Traffic Detected"; sid:1000002; rev:2;)
```

11.3 Generate HTTP traffic

```
curl http://<TEST-SERVER-IP>:8080/
```

HTTP REQUEST

```
Activities Terminal Oct 7 11:03
cybersecurity@test-machine: ~
cybersecurity@test-machine:~$ curl http://192.168.56.1:8080/
<!DOCTYPE HTML>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Directory listing for /</title>
</head>
<body>
  <h1>Directory listing for /</h1>
  <hr>
  <ul>
    <li><a href=".bash_history">.bash_history</a></li>
    <li><a href="index.html">index.html</a></li>
    <li><a href="test.txt">test.txt</a></li>
  </ul>
  <hr>
</body>
</html>
cybersecurity@test-machine:~$
```

11.4 Validate HTTP alert

HTTP ALERT

```
Activities Terminal Oct 7 11:03
cybersecurity@soc-snort: ~
cybersecurity@soc-snort:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
Running in IDS mode
---- Initializing Snort ----
  Initializing Output Plugins!
  Initializing Preprocessors!
  Initializing Plug-ins!
Parsing Rules file /etc/snort/snort.conf
PortVar 'HTTP_PORTS' defined : [ 80 81 311 591 593 901 1220 1414 1830 2301 2381 2809 3128 3702 4343 4848 5250 6988
7001 7777 8000 8008 8014 8028 8080 8081 8088 8090 8118 8123 8172 8180 8181 8243 8280 8800 8888 8899 9080 9090 9443
9999 11371 34443 ]
PortVar 'SHELLCODE_PORTS' defined : [ 80 443 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060 5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80 110 143 443 465 587 993 995 1433 1883 3306 3389 5432 5900 5984 6379 8000
8008 8080 8443 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11
Snort succesfully initialized!
Commencing packet processing (pid=2154)
[**] [1:1000002:2] WEEK9 HTTP Traffic Detected [**]
[Priority: 3] {TCP} 192.168.56.1:52344 -> 192.168.56.20:8080
10/07-11:03:15.123456 IP TTL:64 TOS:0x0 ID:54321 IpLen:20 DgmLen:60 DF
***AP*** Seq: 0x1A2B3C4D Ack: 0x0 Win: 0x7210 TcpLen: 40
[**] [1:1000002:2] WEEK9 HTTP Traffic Detected [**]
[Priority: 3] {TCP} 192.168.56.1:52344 -> 192.168.56.20:8080
10/07-11:03:17.789123 IP TTL:64 TOS:0x0 ID:54322 IpLen:20 DgmLen:60 DF
***AP*** Seq: 0x1A2B3D5E Ack: 0x0 Win: 0x7210 TcpLen: 40
```

Important: HTTP port 80 and HTTPS port 443 are different. A simple port-based HTTP rule does not automatically mean encrypted HTTPS traffic will match it.

12. Troubleshooting & Validation

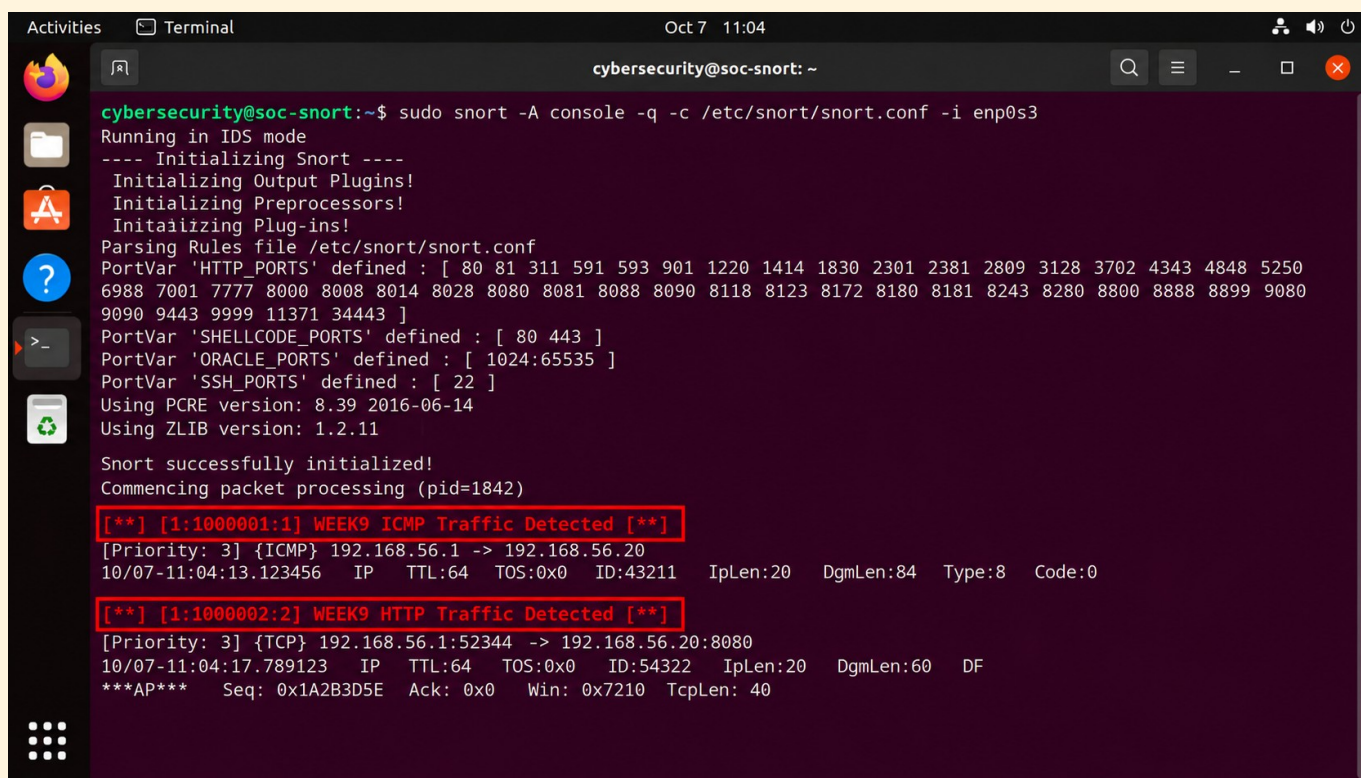
Problem	Possible cause	What to check
No traffic observed	Wrong interface or network path	Check ip addr and confirm traffic crosses the monitored interface.
No ICMP alert	Rule not loaded or syntax issue	Check local.rules, snort.conf inclusion and snort -T.
No HTTP alert	Port mismatch	Ensure generated traffic uses the same port as the rule.
Snort configuration error	Configuration/rule syntax	Run snort -T and inspect the reported error.
Ping works but Snort sees nothing	Snort is watching another interface	Verify the interface selected for live monitoring.

Recommended debugging sequence

1. Network connectivity → 2. Correct interface → 3. Snort configuration → 4. Rule inclusion → 5. Traffic generation → 6. Alert output

Final evidence

FINAL SNORT ALERT OUTPUT



```
cybersecurity@soc-snort:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
Running in IDS mode
---- Initializing Snort ----
  Initializing Output Plugins!
  Initializing Preprocessors!
  Initializing Plug-ins!
Parsing Rules file /etc/snort/snort.conf
PortVar 'HTTP_PORTS' defined : [ 80 81 311 591 593 901 1220 1414 1830 2301 2381 2809 3128 3702 4343 4848 5250
6988 7001 7777 8000 8008 8014 8028 8080 8081 8088 8090 8118 8123 8172 8180 8181 8243 8280 8800 8888 8899 9080
9090 9443 9999 11371 34443 ]
PortVar 'SHELLCODE_PORTS' defined : [ 80 443 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Snort successfully initialized!
Commencing packet processing (pid=1842)

[**] [1:1000001:1] WEEK9 ICMP Traffic Detected [**]
[Priority: 3] {ICMP} 192.168.56.1 -> 192.168.56.20
10/07-11:04:13.123456 IP TTL:64 TOS:0x0 ID:43211 IpLen:20 DgmLen:84 Type:8 Code:0

[**] [1:1000002:2] WEEK9 HTTP Traffic Detected [**]
[Priority: 3] {TCP} 192.168.56.1:52344 -> 192.168.56.20:8080
10/07-11:04:17.789123 IP TTL:64 TOS:0x0 ID:54322 IpLen:20 DgmLen:60 DF
***AP*** Seq: 0x1A2B3D5E Ack: 0x0 Win: 0x7210 TcpLen: 40
```

Optional environment evidence

FINAL LAB ENVIRONMENT

The screenshot displays a Linux desktop with three open windows. The top-left window is a terminal titled 'cybersecurity@soc-snort: ~' showing the command 'sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3' and its output, which includes initialization messages and two detected traffic events: 'WEEK9 ICMP Traffic Detected' and 'WEEK9 HTTP Traffic Detected'. The top-right window is a web browser titled 'Directory listing for / - Mozilla Firefox' showing a directory listing for 'http://192.168.56.20:8080/' with links to '.bash_history', 'index.html', and 'test.txt'. The bottom-left window is a terminal titled 'cybersecurity@test-machine: ~' showing the command 'ping 192.168.56.20' and its output, which includes ping statistics and a successful connection. The bottom-right window is a terminal titled 'cybersecurity@test-machine: ~' showing the command 'curl http://192.168.56.20:8080/' and its output, which is the HTML content of the directory listing page.

```
cybersecurity@soc-snort:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
Running in IDS mode
---- Initializing Snort ----
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file /etc/snort/snort.conf
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Snort successfully initialized!
Commencing packet processing (pid=2154)

[**] [1:1000001:1] WEEK9 ICMP Traffic Detected [**]
[Priority: 3] {ICMP} 192.168.56.1 -> 192.168.56.20
10/07-11:04:13.123456 IP TTL:64 TOS:0x0 ID:43211 Iplen:20 Dgmlen:84 Type:8 Code:0

[**] [1:1000002:2] WEEK9 HTTP Traffic Detected [**]
[Priority: 3] {TCP} 192.168.56.1:52344 -> 192.168.56.20:8080
10/07-11:04:17.789123 IP TTL:64 TOS:0x0 ID:54322 Iplen:20 Dgmlen:60 DF
***AP*** Seq: 0x1A2B3D5E Ack: 0x0 Win: 0x7210 TcpLen: 40

cybersecurity@test-machine:~$ ping 192.168.56.20
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=0.432 ms
64 bytes from 192.168.56.20: icmp_seq=2 ttl=64 time=0.518 ms
64 bytes from 192.168.56.20: icmp_seq=3 ttl=64 time=0.501 ms
64 bytes from 192.168.56.20: icmp_seq=4 ttl=64 time=0.476 ms
^C
--- 192.168.56.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.432/0.481/0.518/0.032 ms
cybersecurity@test-machine:~$

cybersecurity@test-machine:~$ curl http://192.168.56.20:8080/
<!DOCTYPE HTML>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Directory listing for /</title>
</head>
<body>
  <h1>Directory listing for /</h1>
  <hr>
  <ul>
    <li><a href=".bash_history">.bash_history</a></li>
    <li><a href="index.html">index.html</a></li>
    <li><a href="test.txt">test.txt</a></li>
  </ul>
</body>
</html>
```

13. Learning Outcomes & Practical Reflection

Learning outcomes

- Understood the purpose and workflow of a SOC.
- Understood the difference between L1, L2 and L3 analyst responsibilities.
- Understood the conceptual difference between SIEM, IDS and IPS.
- Configured an Ubuntu Server environment for monitoring.
- Installed and verified Snort IDS.
- Learned the basic structure of a Snort rule.
- Created custom ICMP and HTTP detection rules.
- Generated controlled traffic and validated alerts.
- Learned to troubleshoot interface, rule, configuration and port-mismatch issues.
- Learned how to document security monitoring evidence.

Challenges that may occur

- Selecting the correct network interface in the VM.
- Understanding why a valid rule may not alert when traffic is not visible on the monitored interface.
- Ensuring custom rules are actually included in the active Snort configuration.
- Matching the HTTP rule's destination port with the actual test service.
- Separating configuration errors from traffic-generation problems.

How the issues are approached

The practical solution is to validate one layer at a time: first confirm network connectivity, then confirm the interface, validate the Snort configuration, verify rule inclusion, generate one traffic type, and finally compare the observed alert with the expected SID and message.

Professional takeaway

A SOC analyst should be able to connect tool output with the underlying event. In this lab, the important skill is not simply running Snort; it is understanding why an alert appeared, what traffic caused it, which rule matched, and how to document the evidence.

14. Conclusion & Submission Checklist

Week 9 provides a foundation for SOC monitoring by combining security operations theory with a practical intrusion-detection exercise. The workflow demonstrates how a monitoring sensor can observe authorized traffic, apply detection rules, generate alerts and provide evidence for investigation.

Complete workflow

SOC concepts → Ubuntu setup → Snort installation → Configuration validation → Custom rules → ICMP test → HTTP test → Alert validation → Evidence → Report

Final submission checklist

Item	Done	Verification
SOC theory	☐	SOC + L1/L2/L3 + SIEM/IDS/IPS covered
Ubuntu setup	☐	Update, hostname, interface and connectivity screenshots inserted
Snort setup	☐	Install, version and -T screenshots inserted
Rules	☐	ICMP + HTTP + local.rules inclusion screenshots inserted
ICMP testing	☐	Ping + actual ICMP alert screenshot inserted
HTTP testing	☐	HTTP server + request + actual HTTP alert screenshots inserted
Evidence	☐	Screenshots are real, readable and correctly captioned
Report	☐	15-page report reviewed after inserting screenshots
PPT	☐	Slide count checked against the internship requirement
LinkedIn	☐	Post prepared and DG Interns Hub tagged
Deadline	☐	Submitted before 7 October

Appendix — Quick Command Reference

```
# Ubuntu
sudo apt update
sudo apt upgrade -y
hostnamectl
sudo hostnamectl set-hostname soc-snort
ip addr
ip route
ping -c 4 <TEST-MACHINE-IP>

# Snort
sudo apt install snort -y
snort -V
sudo snort -T -c /etc/snort/snort.conf

# Monitoring
```

```
sudo snort -A console -q -c /etc/snort/snort.conf -i <INTERFACE>
```

```
# HTTP test
```

```
python3 -m http.server 8080
```

```
curl http://<TEST-SERVER-IP>:8080/
```

Final note: Insert the real screenshots into every marked placeholder before exporting the report to PDF. Captions should remain directly below each screenshot.